Disjoint sets: merge groups, never split them

The partition changes as groups merge. The query asks: **are these two elements in the same group now?**

Question	Method
How do we identify a set?	A list's set record, or a forest root
How much must a merge change?	Move the shorter list; attach the lower-rank root
Why can later finds get cheaper?	Path compression and amortized sequence analysis

Prerequisites: pointers, rooted trees, asymptotic notation, and the potential method. Rank, level, and iter are defined here.

Three operations maintain one partition

Every created element belongs to exactly one nonempty set; distinct sets are disjoint.

Operation	Precondition	Result
MAKE-SET(x)	x has not been created	Create singleton {x}
FIND(x)	x exists	Return its set's representative
UNION(x,y)	Both elements exist	Merge their sets; do nothing if already together

A representative identifies a set; it need not be its smallest key. Finds on one set return the same representative; different sets have different representatives.

Count calls separately from successful merges

Create n singletons first. Let m count external MAKE-SET, FIND, and UNION calls, so $m \geq n$. Do not count UNION's internal FIND calls again in m.

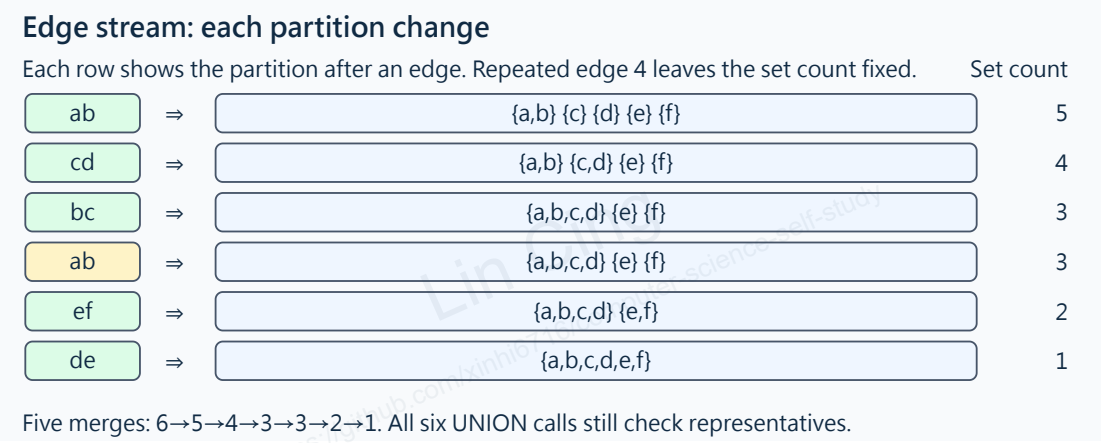
Quantity	Contract
Successful merges	At most n−1, because each removes one set
UNION calls	Repeated calls are allowed; no n−1 bound
LINK(r,s)	Accepts two **distinct roots**, not arbitrary elements

> UNION finds both roots and returns immediately if they agree. **Never LINK a root to itself or increase its rank for a repeated merge.**

FIND may change parents while preserving the partition. Deletion, splitting, and rollback require additional structures and analysis.

Undirected connectivity: what does a new edge merge?

Create a set for each vertex, then UNION each edge. The diagram uses ab, cd, bc, ab, ef, de.



The fourth edge ab is repeated: the call counts, but the partition does not change. Six UNION calls cause five successful merges.

Why are these exactly the connected components?

Invariant: the components of the processed-edge subgraph equal the maintained partition.

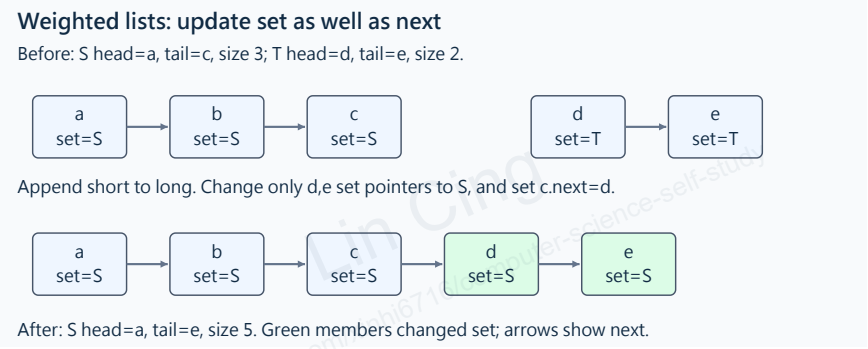
Step	Connectivity	Partition
No edges yet	Each vertex is isolated	One singleton per vertex
New edge within a component	Already connected	UNION changes nothing
New edge between components	Joins exactly those two components	UNION merges exactly their two sets

Induction gives FIND(u)=FIND(v) exactly when an undirected u–v path exists.

DFS/BFS finds static components in $\Theta(n+|E|)$. DSU also supports **connectivity queries as edges arrive**. Ignoring edge directions yields weak components, not SCCs.

Lists: joining tails does not finish the merge

A set record stores head, tail, size. Each member stores next and a set pointer. FIND(x) returns x.set.head in O(1).



> Merging a→b→c with d→e changes tail c→e and size 3→5. **Both d.set and e.set must point to the retained record** before their finds return a.

Finish every step of weighted-list UNION

Let S=x.set and T=y.set. Return for the same record; otherwise retain the longer list as S, with first-set ties.

Order	Update	Purpose
1	S.tail.next=T.head	Join the chains
2	Preserve T's old head; update S.tail and S.size	Keep the traversal start and update metadata
3	Follow T's next links; redirect every member's set to S	Make their finds return S.head
4	Discard T's record only after all redirects	Leave no dangling set pointers

Do not change T's internal next links during traversal. The merge costs $\Theta(1+\min(|S|,|T|))$: **a tail pointer makes joining constant-time, not the whole UNION.**

Why can unweighted lists take quadratic work?

Always append the second set to the first. In UNION(x_2,x_1), UNION(x_3,x_2), …, each new singleton absorbs the existing long list.

Merge	Members whose set pointers change
UNION(x_2,x_1)	1
UNION(x_3,x_2)	2
…	…
UNION(x_n,x_{n-1})	n−1

Total rewrites are n(n−1)/2. There are only n creations and n−1 unions, so m=2n−1, yet the cost is $\Theta(n^2)$.

Moving the shorter list: how often can one member move?

Fix member x. Whenever its set pointer changes, it belonged to the shorter list; its new set is **at least twice as large**.

After k moves its set has at least 2^k members, at most n. Thus $k \leq \lfloor \log_2 n \rfloor$. Sum over members and add the m constant call overheads: **$O(m+n \log n)$**, or $O(m+n \log(n+1))$ including small n.

Quantity	Bound or example
Whole sequence	$O(m+n \log n)$
One merge of two n/2-member sets	Still $\Theta(n)$
Balanced rounds with $n=2^k$	$n/2$ rewrites per round; $(n/2)\log_2 n$ total

> A sequence bound is not a per-call worst-case bound. Weighted union does not make every sequence linear.

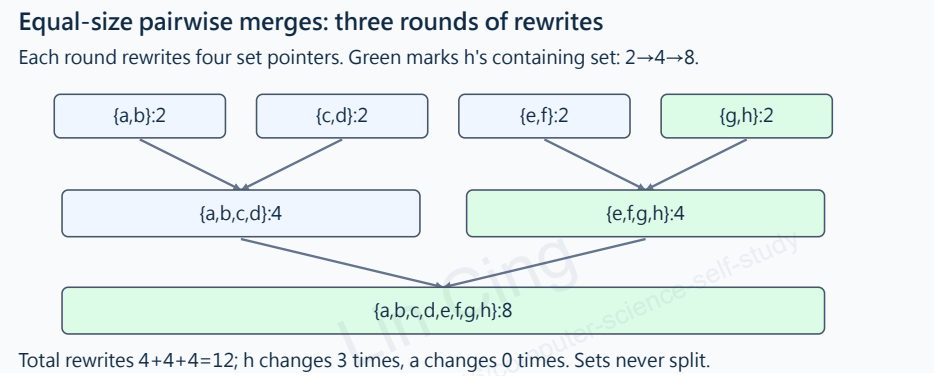
Eight elements: exactly which members move?

Retain the first set on equal sizes. The same operations will later build a forest.

UNION	Old sizes	Members rewritten	New representative / size	Cumulative
a,b	1,1	b	a / 2	1
c,d	1,1	d	c / 2	2
e,f	1,1	f	e / 2	3
g,h	1,1	h	g / 2	4
a,c	2,2	c,d	a / 4	6
e,g	2,2	g,h	e / 4	8
a,e	4,4	e,f,g,h	a / 8	12

Member h follows h→g→e→a as its set grows 1→2→4→8; a never moves. **Twelve total rewrites do not mean every member moves three times.**

One diagram connects round totals to one member's growth



Round totals and one member's move count have different denominators. The table sums to 12 rewrites; h alone moves three times.

Attaching only new singletons to a large set can instead take n−1 rewrites. This meets the upper bound; it does not make every possible sequence linear.

Forests: find the roots before linking

Each element stores parent and rank; a root points to itself. MAKE-SET sets parent[x]=x and rank[x]=0. UNION must first find both roots. Linking arbitrary x directly to y can split an existing set or create a cycle.

```
UNION(x,y):
    r=FIND(x); s=FIND(y)
    if r==s: return r
    return LINK(r,s)
LINK(r,s):                    // r and s are distinct roots
    if rank[r]<rank[s]: swap r,s
    parent[s]=r
    if rank[r]==rank[s]: rank[r]+=1
    return r
```

After linking roots, all old members still reach the new root. One root loses its self-parent, with no reverse path creating a cycle. Unequal ranks leave the retained rank unchanged; equal ranks increase it by one. Rank forests and size-weighted lists are different representations. Rank is not exact set size.

Two-pass FIND: locate the root, then follow the old path

Starting at x, follow parent pointers until reaching a self-parent root. Pass one only finds root r. Pass two redirects each node on the **original search path** directly to r.

```
FIND(x):
    r=x
    while parent[r]!=r: r=parent[r]
    while parent[x]!=x:
        old=parent[x]
        parent[x]=r
        x=old
    return r
```

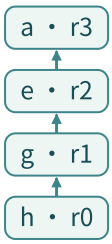
Variable old preserves the next stop before mutation. Using x=parent[x] instead reads the root just written and skips intermediate nodes. Recursive parent[x]=FIND(parent[x]) redirects nodes as calls return.

The two-pass version uses O(1) auxiliary space; the recursive stack scales with original path height. Both take Θ(s) work for s visited nodes, change only this path, and preserve ranks.

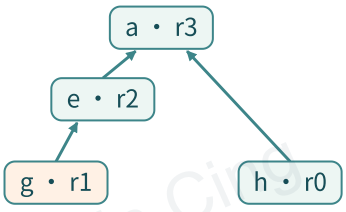
Finding the right root is not full path compression

Arrows point **child→parent**. Only h→g→e→a is drawn; a is self-parent.

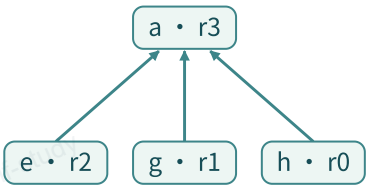
Original path



Jump to root after writing



Save old parent; finish path



Current node in pass two	Save old	Assign parent	Continue at
h	g	a	g
g	e	a	e
e	a	a, unchanged	a, then stop

The same eight elements before and after compression

For the seven earlier unions, equal ranks retain the first root. The successive root ranks are a1,c1,e1,g1,a2,e2,a3. Final parents are b→a, c→a, d→c, e→a, f→e, g→e, h→g; a points to itself.

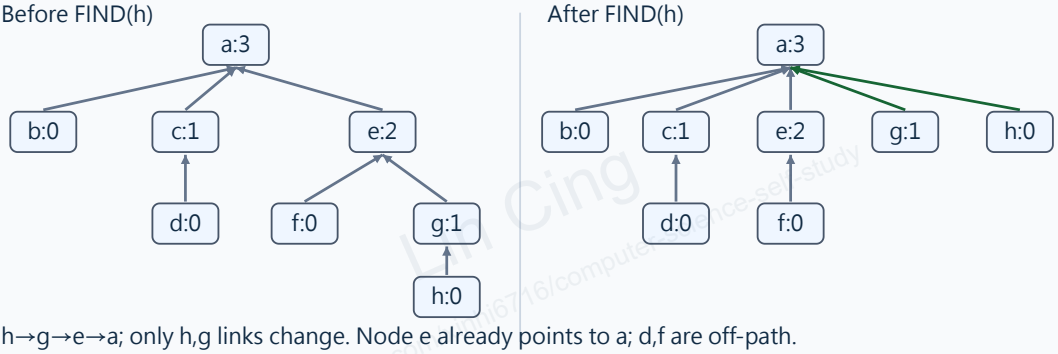
Vertex	a	b	c	d	e	f	g	h
rank	3	0	1	0	2	0	1	0
Parent before FIND(h)	a	a	a	c	a	e	e	g
Parent after FIND(h)	a	a	a	c	a	e	a	a

- Pass one follows h→g→e→a: three parent edges.
- Only **h and g change parent**; e already points to a. Off-path nodes do not change.
- **Every rank stays fixed.** Compression does not replace historical ranks with current heights.

Read the tree: redirect the path, retain the branches

The same eight-node tree: parents before and after compression

Nodes show key:rank; arrows show parent; root self-loops omitted; green links changed.



h→g→e→a; only h,g links change. Node e already points to a; d,f are off-path.

> A later FIND(h) takes one edge; FIND(d) still follows d→c→a. **One FIND compresses only its own search path.**

Why does rank bound tree height?

Rank is an **upper bound**, rather than a measurement of current height. Induct on operations:

Operation	Effect on height	Why height $\leq$ rank survives
Make a singleton	Height is 0	Initial rank is also 0
Merge unequal ranks	Attach the lower-rank root	The rank gap pays for its extra parent edge
Merge equal ranks	Height increases by at most 1	The retained root's rank also increases by 1
Compress a path	Redirect to ancestors, shortening paths	Ranks stay fixed

> **Do not recompute rank after compression.** A rank-2 root may now have height 1.

Rank remembers the members accumulated during merging

A root's rank rises only by merging two different sets of equal rank:

- Rank 0 requires at least one member.
- Two rank-(r−1) sets each contain at least 2^{r-1} members. Their merged rank-r root has at least 2^r.
- Unequal-rank merges add members; FIND preserves set membership.

A nonroot's rank freezes when it loses root status. **Its set then had at least 2^r members.** Thus $2^r \leq n$, giving $r \leq \lfloor \log_2 n \rfloor$. This is not a bound on its current subtree size.

> Rank alone gives $O(\log n)$ per FIND. Compression improves the **amortized sequence bound**; a first deep-leaf query can still traverse $\Theta(\log n)$ edges.

21.4 Fine-grained analysis of rank and compression

Ackermann iteration means composition

Fix the final number of elements n and set $\alpha = \alpha(n)$ throughout the analysis, even while MAKE-SET calls are still creating elements. For $j \geq 1$, define

$$f^{(0)}(j) = j, \quad f^{(i+1)}(j) = f(f^{(i)}(j)), \quad A_0(j) = j + 1, \quad A_k(j) = A_{k-1}^{(j+1)}(j) \ (k \geq 1).$$

The superscript counts **repeated applications to the previous output**. Since $A_0^{(i)}(j) = j + i$, we get $A_1(j) = 2j + 1$. Induction gives $A_1^{(i)}(j) = 2^i(j + 1) - 1$, hence

$$A_2(j) = 2^{j+1}(j + 1) - 1, \qquad A_3(1) = A_2(A_2(1)) = A_2(7) = 2047.$$

Inductively, each A_k is strictly increasing in its input and satisfies $A_k(j) \geq j + 1$: a larger starting input undergoes more iterations, each increasing its input. Since $j + 1 \geq 2$, also $A_k(j) > A_{k-1}(j)$. Both monotonicities are used below.

The chosen inverse-Ackermann convention

Use the single-parameter convention of CLRS third edition, Section 21.4:

$$\alpha(n) = \min\{k \geq 0 : A_k(1) \geq n\}.$$

k	0	1	2	3	4
$A_k(1)$	2	3	7	2047	greater than 10^{80}

Thus $\alpha(2) = 0$, $\alpha(8) = 3$, and $\alpha(2048) = 4$. Other texts may use a two-parameter convention; its numerical values cannot be substituted here.

The complete bound is $O(m(1 + \alpha(n)))$ for m external operations, including n MAKE-SET calls. Only for $n \geq 3$ can it be shortened to $O(m\alpha(n))$. Small instances still require constant work per operation.

Which quantities can change?

Induction on operations shows that a root points to itself and every nonroot has smaller rank than its parent. Linking equal-rank roots raises the retained root's rank by one. Unequal ranks attach the smaller rank under the larger. Compression changes parents, never ranks.

Once a node becomes a nonroot, its rank is fixed. Its parent's rank may increase, or compression may replace the parent by a higher ancestor. Neither event decreases parent rank. This is why a node's level cannot decrease over time.

A rank-r root contains at least 2^r elements. A nonroot retains that historical bound from losing root status. Hence every rank is at most $\lfloor \log_2 n \rfloor < n$, regardless of the current compressed subtree size.

Define level and iter only for positive-rank nonroots. Rank-zero nodes and roots receive separate potential formulas.

Level selects the scale; iter measures progress within it

For a nonroot, write $r = \operatorname{rank}(x) \geq 1$, $p = \operatorname{rank}(\operatorname{parent}(x))$, and define

$$\ell(x) = \max\{k \geq 0 : p \geq A_k(r)\}, \qquad i(x) = \max\{j \geq 1 : p \geq A^{(j)}_{\ell(x)}(r)\}.$$

Here $\ell = \operatorname{level}$ and $i = \operatorname{iter}$. First find the highest scale reached by parent rank; then count iterations at that scale.

Since $p \geq r + 1 = A_0(r)$, we have $\ell \geq 0$. Also $A_\alpha(r) \geq A_\alpha(1) \geq n > p$, so $\ell < \alpha$. The definition gives $i \geq 1$. If $i \geq r + 1$, then

$$p \geq A^{(r+1)}_\ell(r) = A_{\ell+1}(r),$$

contradicting maximality of ℓ. Therefore $0 \leq \ell < \alpha$ **and** $1 \leq i \leq r$.

Nonnegative potential records work still available

Assign a potential to each node and sum over the forest:

$$\phi(x) = \begin{cases} \alpha r, & x \text{ is a root or } r = 0, \\ (\alpha - \ell(x))r - i(x), & x \text{ is a nonroot and } r \geq 1, \end{cases} \qquad \Phi = \sum_x \phi(x).$$

For nonroots, the lower bound is $(\alpha - (\alpha - 1))r - r = 0$, and the upper bound is $\alpha r - 1$. Roots have potential αr; rank-zero nodes have zero potential. Thus Φ is always nonnegative.

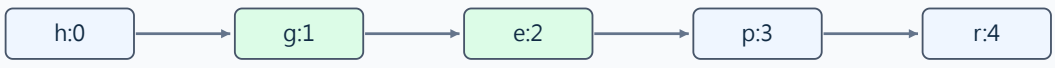
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Potential accounting on a five-node path

Take $n = 16$, $\alpha = 3$, and compress a path of ranks 0, 1, 2, 3, 4. The root keeps its self-parent.

FIND values: progress through level and iter

n=16, α=3; after FIND(h), h,g,e,p all point to r. All ranks stay fixed.



	h:0	g:1	e:2	p:3	r:4
Before (level,iter)					
	—	(0,1)	(0,1)	(0,1)	root
After (level,iter)					
	—	(1,1)	(0,2)	(0,1)	root
	φ 0→0	φ 2→1	φ 5→4	φ 8→8	φ 12→12

Good nodes g,e: path potential 27→25. Exceptions: rank 0, root, last node at each level.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Each good node releases one unit: g gains a level; e increases iter within one level. The other three potentials stay fixed, so the path total falls from 27 to 25.

Why can a rising level reset iter?

Fix the rank r of a nonroot. Within one level, increasing parent rank leaves iter unchanged or increases it. Each additional iteration decreases ϕ by one.

If the level rises from k to $k' > k$, iter may reset from i to a smaller i'. Nevertheless,

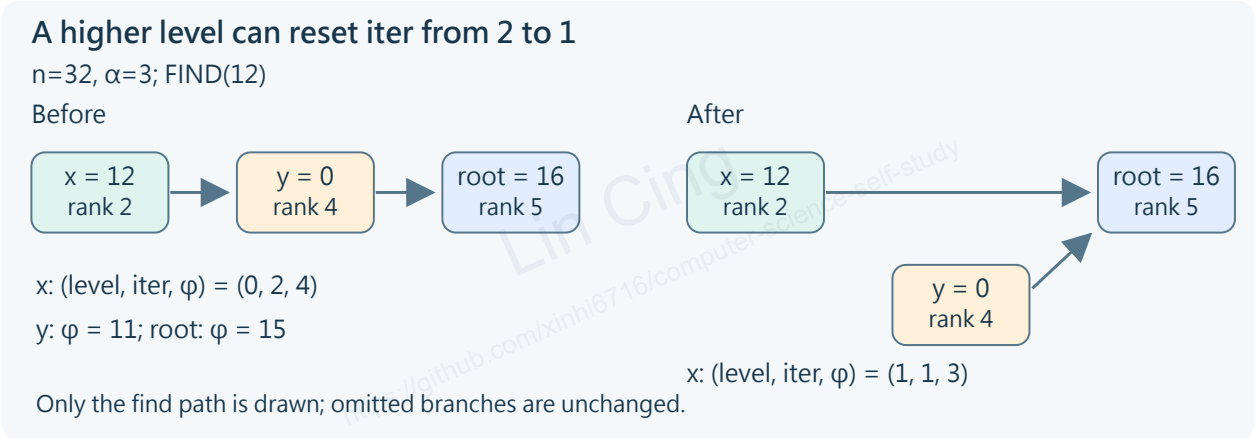
$$\Delta\phi = -(k' - k)r + (i - i') \leq -r + (r - 1) = -1.$$

This uses $k' - k \geq 1$, $i \leq r$, and $i' \geq 1$. **The gain from changing level pays for resetting iter.** The pair progresses in lexicographic order, comparing level first, then iter. Its two coordinates need not increase independently.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A legal FIND that really decreases iter

Take $n = 32$ and $\alpha = 3$. The find path is $12 \to 0 \to 16$, with ranks $2, 4, 5$. Unvisited branches are omitted and remain unchanged.



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Computing the reset example term by term

Node $x = 12$ keeps rank 2. Only its parent rank changes, from 4 to 5.

Quantity	Before compression	After compression
level	$A_0(2) = 3 \le 4 < A_1(2) = 5$, so 0	$A_1(2) = 5 \le 5 < A_2(2) = 23$, so 1
iter	$A_0^{(2)}(2) = 4 \le 4 < A_0^{(3)}(2) = 5$, so 2	$A_1(2) = 5 \le 5 < A_1^{(2)}(2) = 11$, so 1
$\phi(x)$	$(3-0)2 - 2 = 4$	$(3-1)2 - 1 = 3$

Node 0 keeps potential 11, and the root keeps potential 15. All other branches are unchanged. Thus the forest has $\Delta\Phi = (3 + 11 + 15) - (4 + 11 + 15) = -1$. The displayed totals sum only the path; the other 29 nodes need not have zero potential.

Constructing that path with legal operations

Create singleton keys 0 through 31. Equal-rank merges retain the first root.

Operation	Checkable result
Merge adjacent equal-size blocks of 0..15 in rounds	Root 0 has rank 4; node 12 has rank 2
FIND(12)	Parent of 12 becomes 0; ranks stay fixed
Build 16..31 by the same rounds	Root 16 has rank 4
UNION(16,0)	Root 16 rises to rank 5; parent of 0 becomes 16
FIND(12) again	Path $12 \to 0 \to 16$; parent of 12 becomes 16

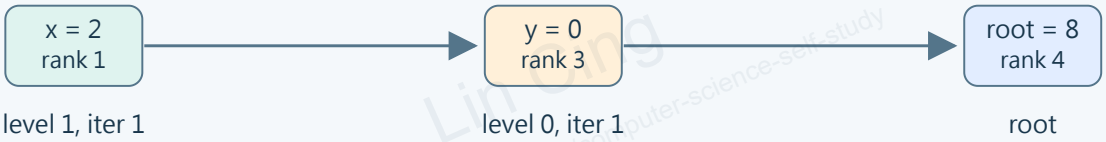
The first half starts with (0,1), (2,3), …, (14,15), then (0,2), (4,6), …, followed by (0,4), (8,12), and finally (0,8). These ranks come from a legal history rather than arbitrary assignments to a drawn chain.

Levels need not increase along a path

Take $n = 16$. Merge 0..7 into root 0 of rank 3, then call FIND(2). Build 8..15 into root 8 of rank 3 and call UNION(8,0). This creates the following legal path.

Increasing ranks do not imply increasing levels

n=16, α=3; FIND(2)



$A_1(1)=3 \le \text{rank}(y) < A_2(1)=7$; $A_0(3)=4 \le \text{rank}(\text{root}) < A_1(3)=7$

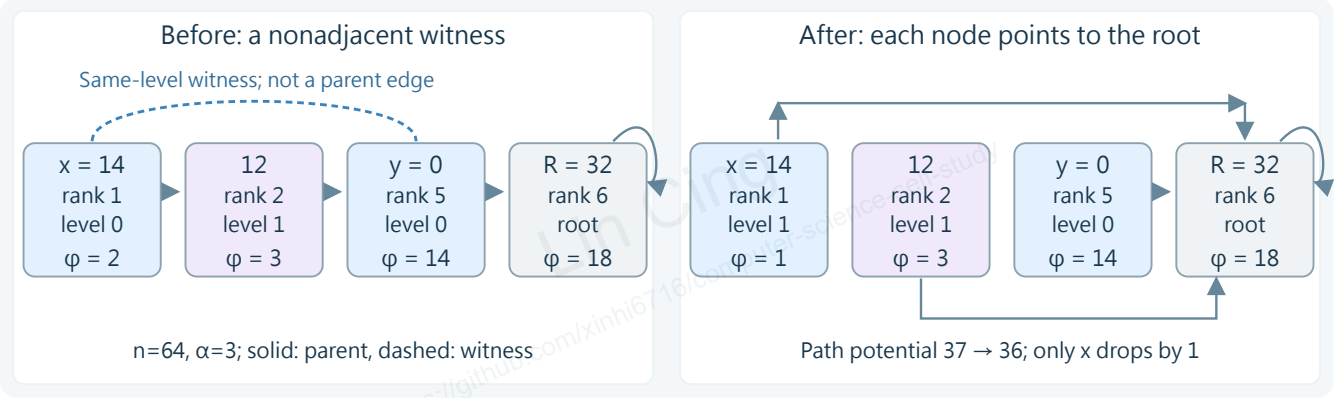
Along this path: ranks 1 < 3 < 4, but levels 1 > 0.

Count the last node of each level; no sorted-level assumption is needed.

Comparing different nodes changes the input rank too. Temporal monotonicity concerns one fixed node.

A same-level witness need not be the next node

A positive-rank nonroot x is good if a later nonroot y on the old path has the same level. **The witness need not be x's parent.**



In 14→12→0→32, levels are 0,1,0. Node 0 witnesses that 14 is good; node 12 may have another level. The root has no level.

Construct the path through legal operations

Start with 64 singletons. Equal-rank UNION retains its first root. Building an interval in rounds means merging adjacent singletons, then adjacent equal-sized blocks.

Operation	Fact established by the history
Build 0..31 in rounds	Root 0 has rank 5; node 12 has rank 2 and node 14 rank 1
FIND(12)	Old path is $12 \to 8 \to 0$; attach 12 to 0, while 14 still points to 12
Build 32..63 in rounds	Root 32 has rank 5
UNION(32,0)	Equal ranks: 32 rises to 6, and 0 points to 32
Before FIND(14)	Path $14 \to 12 \to 0 \to 32$, with ranks 1,2,5,6

FIND(12) follows $12 \to 8 \to 0$ upward to the root, then attaches each visited nonroot directly to that root. It never enters a child subtree, so 14 still points to 12. The later union attaches root 0 to root 32. These ranks follow from legal operations; 32 retains its self-parent.

Compute every level, iteration count, and potential

Fix $n = 64, \alpha = 3$. Level and iter apply only to positive-rank nonroots; the root uses $\phi = \alpha r$.

Node	Rank	Parent rank: old→new	Level: old→new	Iter: old→new	ϕ: old→new
14	1	2→6	0→1	1→1	2→1
12	2	5→6	1→1	1→1	3→3
0	5	6→6	0→0	1→1	14→14
32, root	6	6→6	Not defined	Not defined	18→18

For 14, initially $A_0(1) = 2 \le 2 < A_1(1) = 3$, so level=0,iter=1. Afterwards $A_1(1) = 3 \le 6 < A_2(1) = 7$, and $A_1^{(2)}(1) = 7 > 6$, so level=1,iter=1. Its potential drops from $(3-0)1 - 1 = 2$ to $(3-1)1 - 1 = 1$.

For 12, $5 = A_1(2) \le p < 11 = A_1^{(2)}(2) < A_2(2) = 23$, so level=1,iter=1. For 0, $A_0(5) = 6 \le 6 < A_0^{(2)}(5) = 7 < A_1(5) = 11$, so level=0,iter=1,potential=14. Path total changes $37 \to 36$. All other nodes keep rank,parent rank,root status; **the whole forest also loses exactly one unit**.

The good-node inequality, with a reason for each step

Freeze all quantities before compression. Write $\rho(v) = \mathrm{rank}(v)$, $p_x = \mathrm{parent}(x)$, $p_y = \mathrm{parent}(y)$, and let R be the root. Put $r = \rho(x), k = \ell(x) = \ell(y), i = i(x)$.

Step	Reason
$\rho(R) \geq \rho(p_y)$	p_y is on the root path; rank strictly increases along nonroot parent edges
$\rho(p_y) \geq A_k(\rho(y))$	Definition of $\ell(y) = k$
$A_k(\rho(y)) \geq A_k(\rho(p_x))$	y equals or is above p_x, so $\rho(y) \geq \rho(p_x)$; A_k increases
$A_k(\rho(p_x)) \geq A_k(A_k^{(i)}(r))$	Iter gives $\rho(p_x) \geq A_k^{(i)}(r)$; use monotonicity again
$A_k(A_k^{(i)}(r)) = A_k^{(i+1)}(r)$	One more function composition, not multiplication

The new parent rank is therefore at least $A_k^{(i+1)}(r)$. No step assumes sorted path levels: the proof uses ancestor rank order and monotonicity at one fixed level k.

Crossing a threshold pays for a good node

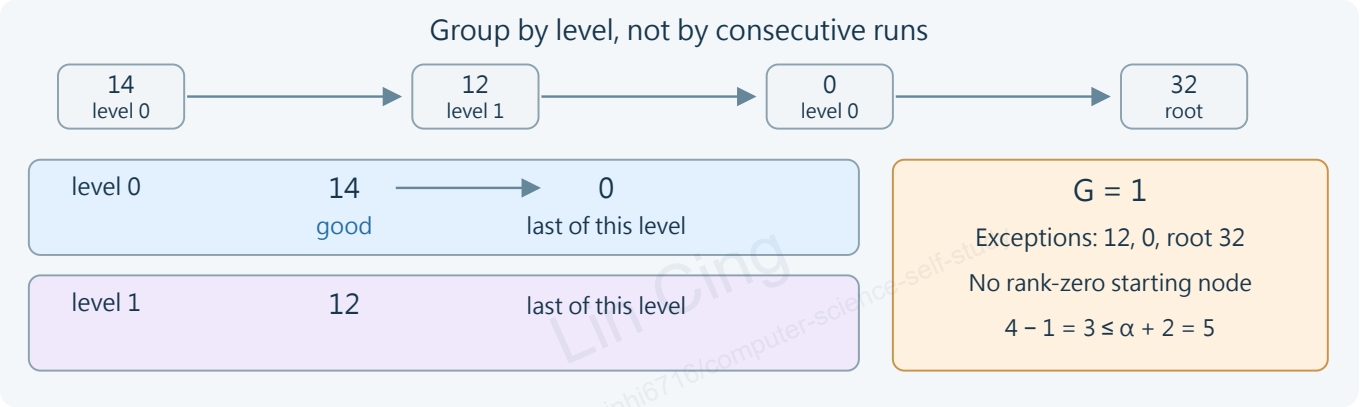
Compression makes R the parent of x, keeping r fixed. The proof crosses the **next iteration threshold of the old level**. It does not guarantee that the new iter exceeds the old one; distinguish two cases.

After compression	Potential change	Why it decreases
Level stays k	$\Delta\phi = -(i' - i) \leq -1$	Iter is at least $i + 1$
Level rises	$\Delta\phi = -(k' - k)r + i - i'$	$k' - k \geq 1, i \leq r, i' \geq 1$, so $\Delta\phi \leq -r + (r - 1) = -1$

In the 64-node example, $x = 14, y = 0, k = 0, i = 1$; the inequalities become $6 \geq 6 \geq 6 \geq 3 \geq 3$. The next old-level threshold is $A_0^{(2)}(1) = 3$. New parent rank 6 also crosses $A_1(1) = 3$, so this is the second case and potential drops by one.

Charge the decrease to good node x, not again to witness y; y's potential need not drop. Other nodes' potentials do not increase, so the good-node decreases may be summed.

Only the last node of each level is an exception



Within one level, every node except the last has a later same-level witness. Here 14 is good; 12 and 0 are the last nodes of levels 1 and 0. Adding root 32 gives three exceptions. Treating consecutive runs 0/1/0 as three groups would miss good node 14.

Convert group counts into total potential decrease

An s-node path has at most α positive-rank nonroot levels, numbered $0, \dots, \alpha - 1$. At most one last node per level, the root, and a possible rank-zero starting node give at most $\alpha + 2$ exceptions. Strictly increasing ranks allow at most one rank-zero node.

$$G \geq g := \max(0, s - (\alpha + 2)), \qquad \Delta\Phi \leq -G \leq -g.$$

G counts actual good nodes; g is only a conservative lower bound. Here $s = 4, \alpha = 3$, hence $g = 0$, yet $G = 1$ and $\Delta\Phi = -1$. A short path need not attain the bound. The next argument uses the general lower bound to pay for long paths.

Fixed constants pay for the entire scan

Suppose two-pass find costs at most cs. Fix $C > c$ and use scaled potential $\Psi = C\Phi$. The amortized cost satisfies

$$\hat{c} = c_{\text{actual}} + \Delta\Psi \le cs - Cg.$$

Path length	Bound after substituting g
$s \le \alpha + 2$	$cs \le c(\alpha + 2)$
$s > \alpha + 2$	$(c - C)s + C(\alpha + 2) \le C(\alpha + 2)$

Thus each FIND has amortized cost $O(1 + \alpha)$. For $s = 7$, $\alpha = 3$, $c = 2$, $C = 3$, the lower bound is $g = 2$, so $\hat{c} \le 14 - 6 = 8$. Keep the same C throughout the sequence.

Only the retained LINK root can gain potential

MAKE-SET adds a rank-zero node with zero potential and still has amortized cost $O(1)$. Suppose LINK attaches old root u under retained root v.

Possibly affected nodes	Reason for the potential change
u	Loses root status without changing rank; potential does not rise
Old children of v	Parent rank cannot fall; the earlier monotonicity argument applies
v	Unequal ranks leave it fixed; equal ranks add one, increasing potential by α

Every other node keeps its rank, parent rank and root status. Therefore $\Delta\Psi \le C\alpha$, and LINK costs $O(1) + C\alpha = O(1 + \alpha)$ amortized. UNION performs two FIND calls and links only distinct roots, expanding into at most three operations.

Recovering actual sequence cost

Let c_j be actual cost and $a_j = c_j + \Psi_j - \Psi_{j-1}$ amortized cost. Summing cancels intermediate potentials:

$$\sum_{j=1}^{m'} c_j = \sum_{j=1}^{m'} a_j + \Psi_0 - \Psi_{m'} \leq \sum_{j=1}^{m'} a_j.$$

The inequality uses the empty initial forest, $\Psi_0 = 0$, and nonnegative final potential. Expanding UNION gives $m' \leq 3m$, so the original m operations cost $O(m(1 + \alpha(n)))$ in total.

This is a worst-case bound on the entire sequence. With rank and compression, a single FIND can still cost $\Theta(\log n)$: build a balanced rank tree and first query a deepest leaf. A linear chain requires removing the rank rule and does not satisfy this theorem's hypotheses.

Potential analysis reference: CLRS, third edition, Section 21.4, printed pp. 576–581.

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study